Mixture of Experts & Numerical Precision

CS290W LLM08 Lecture

LU Yuzhou

Lab Overview

Today's Topics:

1. **Mixture of Experts (MoE)**: Sparse activation for efficient scaling
2. **Numerical Precision**: FP32, BF16, FP16, FP8 trade-offs
3. **Quantization**: INT8/INT4 for model compression

Why This Matters

Modern LLMs (Mixtral, DeepSeek-V3, Qwen3) use MoE to scale to hundreds of billions of parameters while keeping inference practical.

- **Recommended Hardware**: AMD Ryzen AI Halo Processors
- **Environment**: ROCm + PyTorch via AUP Learning Cloud

The Scaling Dilemma

As language models grew, researchers faced a fundamental challenge:

Dense Models Don't Scale Efficiently

- Larger models $\rightarrow$ better quality but proportional compute cost
- Training 100B+ dense model: prohibitively expensive
- Inference: every token requires ALL parameters

Example: A 100B parameter dense model:

- Memory: 400 GB (FP32)
- Compute: 100B ops per token
- Inference speed: impractically slow

Solution: Decouple **model capacity** from **compute cost**

Mixture of Experts: Core Idea

Key Insight: Not all knowledge needs to be activated for every token.

- Token "python" $\rightarrow$ activate programming experts
- Token "parrot" $\rightarrow$ activate biology experts

MoE Architecture

Instead of one large FFN, use **multiple smaller FFNs** ("experts") plus a **router** that selects which expert(s) to activate.

Key Equation:

$$y = \sum_{i=1}^{M} G(x)_i \cdot E_i(x)$$

where $G(x)$ is gating function, E_i is expert i.

Sparse Activation: Top-K Gating

Sparsity Mechanism:

$$G(x) = \text{softmax}(\text{TopK}(x \cdot W_g))$$

- Only k experts activated per token (typically $k = 1$ or 2)
- **Model capacity** scales with M (number of experts)
- **Compute cost** stays at $k\times$ single-expert cost

Example: 8 Experts, Top-2

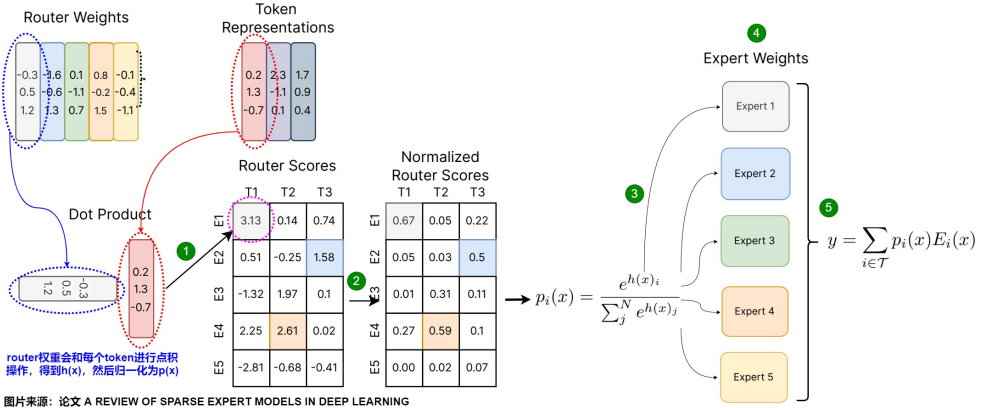
- Total parameters: 8B (same as dense)
- Active parameters: 2B (25%)
- Compute: $4\times$ less than dense!

Router Steps:

1. Linear projection: $x \cdot W_g$ ($O(d \cdot E)$)
2. Top-K selection: find k largest ($O(E \log E)$)
3. Softmax normalization: over k values ($O(k)$)

Gating Network: Token-to-Expert Routing

$$G(x) = Softmax(\quad x \quad \times \quad W)$$



图片来源：论文 A REVIEW OF SPARSE EXPERT MODELS IN DEEP LEARNING

Gating Network: Token-to-Expert Routing (Cont'd)

Routing Process:

1. Input tokens (colored) are processed by router
2. Router computes weights (W) for each expert
3. Dot product produces routing scores $h(x)$
4. Softmax normalizes to probabilities $p(x)$
5. Top-K experts selected per token

Key Point

Each expert receives only the tokens it's specialized to process, enabling efficient computation.

MoE vs Dense: 8B Model Comparison

Metric	Dense 8B	MoE 8B (8 experts, top-2)
Total Parameters	8B	8B
Active Parameters	8B (100%)	2B (25%)
Memory Footprint	32 GB (FP32)	32 GB (FP32)
Compute per Token	8B ops	2B ops (4× less)
Inference Speed	1×	3-4× faster*

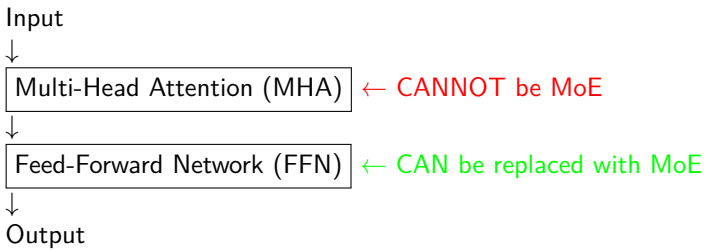
Key Advantage

MoE achieves GPT-level quality with **manageable inference cost** by activating only a subset of parameters.

*Speedup depends on implementation efficiency and memory bandwidth

Where Does MoE Fit in Transformer?

Standard Transformer layer:



Why MHA Cannot Be MoE:

- **Token Mixing:** Each query must attend to ALL keys
- **Context Dependency:** Output for token i depends on all tokens j
- **Mathematical Structure:** QK^T creates $N \times N$ attention matrix
- Sparse activation would break global context flow

LLaMA-Style FFN Expert: SwiGLU

Each expert uses the **SwiGLU** (Swish Gated Linear Unit) activation:

Standard FFN (pre-LLaMA)

$$\text{FFN}(x) = W_2 \cdot \text{ReLU}(W_1 \cdot x)$$

LLaMA FFN with SwiGLU

$$\text{FFN}(x) = W_{\text{down}} \cdot \Big(\text{SiLU}(W_{\text{gate}} \cdot x) \otimes (W_{\text{up}} \cdot x)\Big)$$

where $\otimes$ is element-wise multiplication.

Layer	Shape	Purpose
gate_proj	$d \to d_{\text{inter}}$	Produces gate activations
up_proj	$d \to d_{\text{inter}}$	Produces value activations
down_proj	$d_{\text{inter}} \to d$	Projects back to hidden dim
SiLU	–	$\text{SiLU}(x) = x \cdot \sigma(x)$

Router Compute: Is It Expensive?

Router computes: $G(x) = \text{softmax}(\text{TopK}(x \cdot W_g))$

Compute Breakdown per Token:

1. Linear projection $x \cdot W_g$: $O(d \cdot E)$
2. Top-K selection: $O(E \log E)$
3. Softmax over k values: $O(k)$

Example: $d = 4096$, $E = 8$, $k = 2$

- Router: 32K ops/token
- Expert FFN: 131M ops/token
- **Router overhead: 0.02%** (negligible!)

Combine Layer Cost

Combining expert outputs: $O(k \cdot d)$ per token
For $k = 2$, $d = 4096$: negligible compared to expert compute.

MoE Penalties vs Dense Models

Despite compute savings, MoE introduces challenges:

Penalty	Description	Impact
Communication	All-to-all in distributed training	10-30% slowdown
Load Imbalance	Uneven expert utilization	Wasted compute
Router Compute	Gating network overhead	5% (small)

Load Imbalance Problem:

- Without constraints, router tends to favor few "popular" experts
- Some experts over-trained, others under-trained
- GPU load imbalance in distributed training/inferring

Solution: Add **load-balancing auxiliary loss**

Load Balancing Loss

Switch Transformer Auxiliary Loss:

$$\mathcal{L}_{\text{balance}} = \alpha \cdot N_E \cdot \sum_{i=1}^{N_E} f_i \cdot P_i$$

Where:

- f_i = fraction of tokens routed to expert i
- P_i = average router probability for expert i
- N_E = number of experts

How It Works

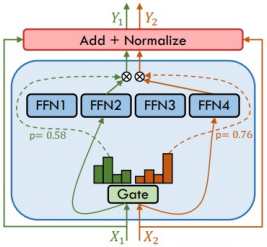
The loss penalizes:

- Experts that receive too many tokens (high f_i)
- Experts with high average probabilities (high P_i)

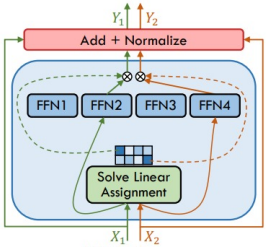
Result: encourages **uniform expert utilization**.

Note: DeepSeek-V3 uses shared experts with capacity constraints instead.

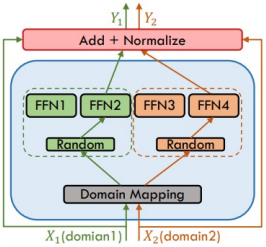
Gating Functions: Different Routing Strategies



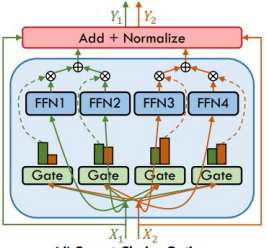
(a) Sparse MoE (Top-1 Gating)



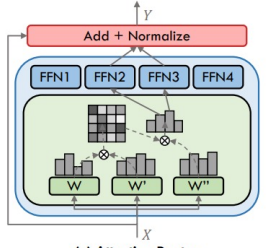
(b) BASE Layers



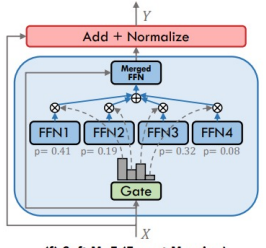
(c) Domain Mapping + Random Gating



(d) Expert-Choice Gating



(e) Attention Router



(f) Soft MoE (Expert Merging)

Gating Functions: Different Routing Strategies(Cont.)

Common Gating Strategies:

- **(a) Top-1 gating:** Select single best expert (Switch Transformer)
- **(b) BASE layer:** Balanced assignment with soft weighting
- **(c) Stochastic gating:** Random selection with domain mapping
- **(d) Expert selection:** Multiple experts per token
- **(e) Attention router:** Attention-based routing scores
- **(f) Soft MoE:** All experts with learned merging

Notable MoE Models in Production

Model	#Experts	Top-K	Notes
Mixtral 8×7B	8	2	Popular open-source MoE
Mixtral 8×22B	8	2	Larger experts
DeepSeek-V3	256+1	8+1	Shared expert + routing
Qwen3-235B-A22B	128	8	Alibaba open-source

DeepSeek-V3 Configuration

- Total: 670B parameters, 37B active
- 256 routed experts + 1 shared expert
- Top-8 from routed + always use shared
- Achieves GPT-4 level quality with efficient inference

Floating-Point Format Comparison

Format	Sign	Exp	Mantissa	Range	Use Case
FP32	1	8	23	$\pm 3.4 \times 10^{38}$	Full-precision training
BF16	1	8	7	$\pm 3.4 \times 10^{38}$	Large-scale training
FP16	1	5	10	$\pm 6.55 \times 10^{4}$	Mixed-precision
FP8 E4M3	1	4	3	± 448	Latest GPU inference
FP8 E5M2	1	5	2	$\pm 5.7 \times 10^{4}$	Latest GPU training

Trade-offs

- **More exponent bits** $\rightarrow$ wider dynamic range
- **More mantissa bits** $\rightarrow$ higher precision
- BF16: same range as FP32, less precision (good for training)
- FP16: higher precision but narrow range (overflow risk)

Mixed Precision Training

Idea: Combine FP32 (master weights) with FP16/BF16 (compute)

Benefits

- **Halve** memory usage
- **Double** throughput on Tensor Cores
- Maintain FP32-level training accuracy

Typical Flow:

1. Forward pass in FP16/BF16
2. Loss computed in FP32 (with loss scaling for FP16)
3. Backward pass in FP16/BF16
4. Weight update in FP32 (master copy)

Memory Savings

For a 1B parameter model:

- FP32: 4 GB
- FP16/BF16: 2 GB (**50% savings**)

Quantization: Model Compression

Goal: Map floating-point weights to lower-bit integers.

Symmetric Quantization Formulas

$$\text{scale} = \frac{\max(|x|)}{q_{\max}}$$

$$x_q = \text{round}\left(\frac{x}{\text{scale}}\right), \text{clipped to } [q_{\min}, q_{\max}]$$

$$x_{\text{dequantized}} = x_q \cdot \text{scale}$$

Where:

- $q_{\min} = -(2^{\text{bits}-1})$ (e.g., -128 for INT8)
- $q_{\max} = 2^{\text{bits}-1} - 1$ (e.g., 127 for INT8)

Quantization: Size vs Accuracy Trade-off

Format	Bits	Size Reduction	Typical Error
FP32	32	1×	0.000000
INT8	8	4×	0.001-0.01
INT4	4	8×	0.01-0.1

Example: 1B Parameter Model

- FP32: 4 GB
- INT8: 1 GB (**4× smaller**)
- INT4: 0.5 GB (**8× smaller**)

Use Cases:

- INT8: Production inference (minimal accuracy loss)
- INT4: Edge deployment, mobile (acceptable accuracy loss)

Lab: What You'll Implement

Step 1: Top-K Gating Network

```
class TopKGating(nn.Module):
    def forward(self, x):
        scores = self.router(x)            # (B, S, E)
        topk_scores, topk_idx = torch.topk(scores, k)
        probs = torch.softmax(topk_scores, dim=-1)
        gate = torch.zeros_like(scores)
        gate.scatter_(-1, topk_idx, probs)  # sparse gate
        return gate, topk_idx
```

Step 2: LLaMA-Style Expert

```
class Expert(nn.Module):
    def forward(self, x):
        return self.down_proj(
            self.act_fn(self.gate_proj(x)) * self.up_proj(x)
        )
```

Step 3: Assemble MoE Layer

- Route tokens to experts
- Combine weighted outputs
- Compute load balancing loss

Lab: Quantization Exercise

Implement Symmetric Quantization:

```
def quantize_tensor(x, num_bits=8):
    qmin = -(2 ** (num_bits - 1))
    qmax = 2 ** (num_bits - 1) - 1
    scale = x.abs().max() / qmax
    x_q = torch.clamp(torch.round(x / scale), qmin, qmax)
    return x_q, scale

def dequantize_tensor(x_q, scale):
    return x_q.float() * scale
```

Experiments:

- Quantize MLP weights to INT8 and INT4
- Measure reconstruction error
- Compare FP32 vs quantized model output

Expected Results

INT8: mean abs error 0.001
INT4: mean abs error 0.01 (higher, but 8$\times$ compression)

Summary: Key Takeaways

1. **MoE Architecture**: Sparse activation via Top-K gating
   - Scales capacity without scaling compute
   - Only FFN layer can be MoE (not attention)
2. **Router**: Linear + TopK + Softmax + scatter
   - Negligible overhead (0.02%)
   - Creates sparse gate with k nonzero values
3. **Load Balancing**: Auxiliary loss prevents expert collapse
4. **Floating-Point**: BF16 (wide range), FP16 (higher precision)
5. **Quantization**: INT8/INT4 for inference compression
   - Symmetric quantization: $x_q = \text{round}(x/\text{scale})$
   - Trade-off: size vs accuracy

Experiment Further

- Modify `top_k` from 2 to 1: observe expert distribution change
- Compare per-channel vs per-tensor quantization accuracy
- Apply quantization to real LLaMA model using `bitsandbytes`
- Implement capacity factor to limit max tokens per expert
- Benchmark MoE vs dense FFN with same compute budget

Resources

- **Mixtral Paper**: arXiv:2401.04088
- **Switch Transformer**: arXiv:2101.03961
- **DeepSeek-V3**: arXiv:2412.19437
- **Lab Notebook**: `LLM08-MoE-and-Numerical-Precision.ipynb`